Project Security Architecture

This document outlines the core security tools and methodologies integrated into the application to protect sensitive data, prevent malicious inputs, and enforce strict execution boundaries.

Note on AI Engine: Currently, this system uses Google's Gemini models for its core orchestration and AI interactions (referred to herein as Siraugga). However, the architecture is designed to be model-agnostic and the underlying AI engine may be swapped out for other applications or LLMs in future iterations.

1. Data Loss Prevention (DLP) & PII Redaction

Tool/Library: Microsoft Presidio (powered by spaCy NLP)

Purpose in the Application: This tool acts as a real-time data privacy filter. Whenever text flows through the application (such as incoming chat prompts or outgoing AI responses), the system uses Natural Language Processing (NLP) to detect sensitive information. This includes Personally Identifiable Information (PII) like email addresses, phone numbers, API keys, and other internal secrets.

Action: The system dynamically scrubs and anonymizes detected entities (e.g., replacing a real email with `<EMAIL_ADDRESS>`) before the data reaches its final destination or is displayed on the screen. This ensures continuous compliance and prevents accidental data leaks.

---

2. Prompt Injection Defense

Methodology: Semantic Delimiters / XML-Tag Context Isolation

Purpose in the Application: To prevent attackers or poisoned third-party assets from hijacking Siraugga's underlying instructions (a vulnerability known as Prompt Injection).

Action: When the application reads or processes untrusted content (such as community mods, external game assets, or user-provided scripts), it isolates this data from the core system prompts. It does this by wrapping the untrusted content in strict semantic delimiters (e.g., `<user_data_untrusted> ... </user_data_untrusted>`). This structural boundary explicitly instructs Siraugga to treat the enclosed content strictly as passive data rather than executable commands, neutralizing any hidden malicious directives.

---

3. Role-Based Access Control (RBAC) & Sandboxing

Methodology: Multi-Tiered Authorization & Canonical Path Resolution

Purpose in the Application: To enforce a zero-trust boundary that ensures users only interact with data appropriate for their specific clearance level (e.g., Administrator, Developer, Modder).

Action:

- **Symlink Traversal Defense**: The application dynamically resolves all requested filesystem operations to their absolute, physical locations. This prevents malicious actors from using shortcut links or path traversal techniques (`../../`) to escape their designated workspace and access sensitive host files.
- **Access Tiers**: Content is classified into strict security tiers. Administrators retain full visibility and execution rights across the system. Conversely, unprivileged users are tightly confined to designated "safe workspaces." System configurations, execution logs, and orchestration logic remain entirely invisible and inaccessible to non-admin roles.

---

4. Zero-Trust AI & Immutable Backend Truth

Methodology: Server-Side Session Enforcement

Purpose in the Application: To guarantee that Siraugga itself cannot be socially engineered into bypassing security restrictions on behalf of a malicious user.

Action: The architecture operates under a "Zero-Trust" model for Siraugga (The Autonomous DevTool). If a user attempts to elevate their privileges through conversation (e.g., claiming to be an administrator in the chat), Siraugga might acknowledge the prompt, but it is physically incapable of complying. All sensitive actions (reading files, executing commands) are intercepted and verified by the backend server using the user's immutable authentication token. The AI has no capacity to alter backend roles, and the server, not Siraugga, holds the ultimate cryptographic truth.

---

5. Error Message Sanitization (CWE-209 Prevention)

Methodology: Generic Exception Handling

Purpose in the Application: To prevent the leakage of sensitive internal system information (such as backend logic, file paths, or security configurations) to an attacker.

Action: When a user or Siraugga violates a security boundary, the backend intercepts the action and returns a highly generic error string (e.g., "Permission Denied: Access outside of project workspace is forbidden"). The AI only receives this generic string. Consequently, if a user asks Siraugga *why* it was blocked, Siraugga is physically incapable of revealing the underlying logic, security variables, or the true filesystem structure, because that data was intentionally excluded from its memory.

---

6. Directory Obfuscation & Path Canonicalization

Methodology: Logical Boundaries & Realpath Verification

Purpose in the Application: To ensure that users restricted to a single working directory cannot map out the underlying host operating system or escape their designated boundaries.

Action:

- **Obfuscation**: The application presents paths to unprivileged users and Siraugga in a

logical, relative format. The true absolute directory path (e.g., `C:\Users\...\` or `/mnt/c/...`) of the host's underlying operating system is kept completely obfuscated.
- **Escape Prevention**: If a malicious user attempts to "guess" their way out by instructing Siraugga to use deep path traversal techniques (such as `../../../../../etc/passwd` or `..\..\Windows`), the backend's path canonicalization algorithm calculates the true physical destination. If that destination falls anywhere outside the pre-approved anchor directory, the request is instantly destroyed. It is mathematically impossible to traverse upwards beyond the anchor root.

---

7. Financial DoS & API Quota Protection

Methodology: Server-Side Rate Limiting

Purpose in the Application: To prevent malicious actors from spamming the system with high-frequency automated requests designed to exhaust LLM API quotas, inflate billing, or crash the event loop (Financial Denial of Service).

Action: The system tracks incoming requests per user session using a token-bucket algorithm. If an unprivileged user exceeds a strict, pre-defined maximum number of requests per minute, the server instantly drops the request. This completely neutralizes automated script spam while maintaining smooth access for normal users.

---

8. Time-of-Check to Time-of-Use (TOCTOU) Prevention

Methodology: Strict File Descriptors (`O_NOFOLLOW`)

Purpose in the Application: To close the race-condition window where a highly sophisticated attacker might attempt to bypass security by swapping a legitimate file with a malicious symlink in the exact millisecond between the server's path validation check and the actual file read/write operation.

Action: Rather than relying purely on standard string-based filesystem operations, the backend reads and writes files using direct operating-system level file descriptors wrapped in strict anti-symlink flags. If the final destination is swapped to a symlink at the very last millisecond, the OS kernel instantly aborts the operation, making it mathematically impossible to exploit the race condition.

---

9. Second-Order Execution (Trojan Horse) Defense

Methodology: Static Code Analysis Interception

Purpose in the Application: To prevent Siraugga from being tricked into writing malicious code (e.g., reverse shells, destructive commands) into game scripts that the host Game Engine might blindly execute at a later time.

Action: Even though Siraugga itself is securely sandboxed, any code it generates could be dangerous if executed by a secondary system. Before any executable file (such as `.lua`, `.py`, or `.sh` scripts) is actually saved to the disk, the backend intercepts the payload and performs a static code analysis scan. If high-risk system commands (e.g., `os.execute`, `subprocess`, `rm -rf`) are detected, the payload is destroyed and the write operation is permanently

blocked.

---

10. Sub-File Granularity & AST Diffing (The Raugus NGPF)

Methodology: Next-Gen Prompt Firewall & Abstract Syntax Tree Parsing

Purpose in the Application: To solve the dilemma of conflicting permissions where an AI has legitimate `Write` access to a file, but specific critical Functions or Variables inside that file must remain strictly `Read-Only`.

Action: The architecture utilizes a proactive memory-hook (`PreToolUse`) to intercept any modification attempts before they reach the OS file system.

1. The **Raugus Map Generator** uses Python's `ast` library to deep-scan source code, assigning independent read/write/execute permissions to every top-level Function and Variable.
2. When the AI attempts to rewrite code (e.g., via `replace_file_content`), the engine halts execution.
3. The hook loads the original file into RAM, virtually applies the AI's proposed replacement code, and parses the new Abstract Syntax Tree (AST).
4. By cross-referencing the modified line ranges, the NGPF mathematically identifies exactly which Functions the AI is attempting to overwrite.
5. If the AI is modifying a locked `Read-Only` Function, the hook instantly denies the tool execution. The physical file on disk remains 100% untouched.
6. **Elegant Degradation**: Instead of violently crashing the session, the Hook returns the NGPF error directly into the AI's context window. The AI remains alive, gracefully informs the user of the security block, and pivots to writing a manual Implementation Plan.

---

11. The Semantic Mantrap (Concurrency Firewall)

Methodology: Cross-Platform OS-Level File Locking (`fcntl` / `msvcrt`)

Purpose in the Application: To prevent Agentic Race Conditions. When an autonomous AI swarm executes multiple subprocesses simultaneously, there is a risk that two agents might attempt to modify a core database or bypass the AST Diffing engine at the exact same nanosecond.

Action: The architecture routes high-risk modifications through a "Semantic Mantrap."

1. The framework demands an exclusive OS-level lock on the target logic flow.
2. If Subagent A is currently modifying a file, Subagent B's thread is physically frozen by the Operating System Kernel at the "Outer Door".
3. This staggers the concurrency, forcing agents into a strict, one-by-one queue.
4. The Mantrap dynamically auto-detects the host OS, seamlessly pivoting between `fcntl.flock` (for Linux/WSL) and `msvcrt.locking` (for Native Windows) to ensure cross-platform thread safety without crashing.

---

12. The Immutable Host Doctrine (Containerization)

Methodology: Read-Only Filesystems & Ephemeral RAM-Disks (Docker)

Purpose in the Application: To ensure that if Siraugga Agent or the Application Server is ever fully compromised (e.g., via Remote Code Execution), the attacker cannot persist their presence, install rootkits, or exfiltrate sensitive files.

Action: The entire backend operates inside a Docker Container engineered with absolute Zero-Trust primitives:

- `read_only: true`: The core application filesystem cannot be modified, preventing code injection or script overwriting.
- `tmpfs`: Writable areas (like logs or scratch directories) are mapped exclusively to volatile RAM-disks. Upon container restart, all data ceases to exist, mathematically neutralizing Data Remanence and ensuring a pristine environment on every boot.
- Compute Limits: Hardcaps on RAM and CPU (`deploy.resources.limits`) prevent Cryptojacking or CPU exhaustion attacks.

---

11. The Semantic Firewall (Evaluator LLM)

Methodology: The Interceptor Pattern / Middle-ware Intent Classification

Purpose in the Application: To defend against advanced Prompt Injection and "Jailbreak" attacks that attempt to psychologically manipulate Siraugga into overriding its own System Prompts or RBAC rules.

Action: Before an untrusted user prompt is ever processed by the Main Agent, it is intercepted and routed to a dedicated, stateless, ultra-fast "Evaluator Model" (e.g., NeMo Guardrails or a fast-inferencing LLM). This Semantic Firewall acts as a strict classifier. If it detects adversarial intent, prompt injection signatures, or rule-override attempts, the firewall drops the connection instantly (Fail-Closed) and returns a 403 Forbidden. The Main Agent is fully shielded from the hostile semantic payload.

---

12. Capability Bootstrapping (Ephemeral UI Plugins)

Methodology: Iframe Sandboxing & Volatile Media Projection

Purpose in the Application: To allow Siraugga Agent to generate dynamic, interactive User Interfaces (e.g., Exam Portals, Video Players) without granting it write access to the core frontend application.

Action: Instead of modifying the master application DOM (which risks Cross-Site Scripting or Template Corruption), Siraugga utilizes pre-approved "Skills" to generate UI components ephemerally.

1. Siraugga duplicates a read-only blueprint.
2. It injects the custom content (e.g., an accessible YouTube iframe).
3. It saves the resulting file to a volatile `/static/scratch/` RAM-disk.
4. Siraugga returns an isolated `<iframe src="/static/scratch/...">` to the user's chat stream.
5. An OS-level Reaper Daemon automatically shreds the temporary file upon session expiration.

---

13. Subagent Zombie Reaping (Resource Exhaustion Defense)

Methodology: Process Group Lifecycle Management (`os.killpg`)

Purpose in the Application: To prevent "Agentic Drift" and catastrophic resource exhaustion caused by orphaned Subagents that detach from the primary UI layer and run infinitely in the background, consuming API budgets (Denial of Wallet).

Action: The Main Agent is initialized within an isolated Linux Session (`start_new_session=True`). Any parallel Subagents it spawns automatically inherit this identical Process Group ID. When the user cancels a task, or an API timeout occurs, the server does not target individual processes. It issues a `SIGKILL` to the entire Process Group, commanding the Linux Kernel to simultaneously and violently terminate the entire process tree. Zero zombie processes can survive.

---

14. Raugus Map VFS (Virtual File System)

Methodology: Cryptographic Path Aliasing & File DNS

Purpose in the Application: To perfectly obfuscate the host operating system's true directory structure from lower-tier users, developers, and autonomous agents, neutralizing Path Traversal attacks and enforcing strict micro-segmentation.

Action: Physical file paths (e.g., `/mnt/c/Users/...`) are never exposed or accepted by the backend. Instead, the architecture utilizes an internal "Phonebook" mapping database. Users request access to abstract aliases (e.g., `system.backend.core`). The server's Resolver intercepts the alias, verifies the user's RBAC Tier against the Phonebook, and silently resolves the true physical path on the backend. This allows System Admins to easily provision highly contained virtual sandboxes for dev teams without moving physical files.

5. Agentic Sanitization & UI Architecture

Methodology: Tier-Driven LLM Summarization & 3D Flip UI

Purpose in the Application: To ensure lower-tier users can understand technical implementations and system changes without ever being exposed to raw source code, backend blueprints, or root credentials.

Action: The system employs a dynamic, fully-decoupled Jinja2 frontend powered by a centralized `ui_config.json`. The chat interface features 3D Flip Cards for artifact inspection and universal Code Copy buttons. To enforce zero-trust security on the frontend, the system relies on **Agentic Sanitization**. Instead of hardcoded regex filters, lower-tier users requesting access to implementation plans trigger automated background prompts. These prompts force the AI to dynamically downgrade, redact, and summarize the document to match the user's specific clearance tier, while completely blocking raw code access for Guest accounts.

The Siraugga Baseline Configurations (Governance &

Policy)

A baseline is not just a technical setting; it is the very foundation of discipline and principles. Baselines form the absolute starting point of our governance and policies. Without a mathematical baseline, it is impossible to define discipline, and without discipline, you cannot govern security policies.

In our architecture, the established baselines are non-negotiable and are hardcoded directly into the core orchestration engine and security routing logic:

1. **The Working Directory Baseline (CWD)**: When a non-admin entity (Tier 1-3) connects, their execution baseline is anchored strictly to `/sandbox/projects/<assigned_project_id>`.
2. **The Agent Execution Baseline**: Any autonomous AI invoked within the framework is forcefully injected with the `--sandbox` flag.
3. **The RBAC Authorization Baseline**: Every session maps mathematically to an established tier: `admin` -> Tier 5, `dev` -> Tier 3, `mod` -> Tier 2.
4. **The Network Continuity Baseline**: The WebSocket server enforces a brutal 4-second keep-alive heartbeat to prevent orphaned execution loops.

Security Update: CSP Telemetry & Accessible Video Player

The Zero-Trust Architecture has been enhanced with a **CSP Telemetry Pipeline**. All cross-origin blocks enforced by `security_headers.py` are now automatically reported by the browser and logged to `csp_violations.log`. Additionally, the cross-origin sandboxing issues blocking the Accessible Video Player inside the chat feed have been fully resolved by fixing the CSP whitelist and injecting parent-origin delegation mechanisms (`allow="autoplay"`, `origin: window.location.origin`).